

Swapping the Engine for Autodesk

If the design engine were Autodesk Inventor (or AutoCAD) instead of the Python engine — integration approaches and trade-offs

1 The key point: almost nothing above the engine changes

The system was built so the engine sits behind a **tool boundary**. The agent calls an MCP tool; the MCP tool calls the store; the store calls `engine.generate(params)`, which returns figures plus a drawing file. Everything above that line — the Gemini agent loop, the schema cleaning, the chat endpoint, the entire email propose/confirm flow — only ever speaks in terms of parameters in, figures and a drawing file out.

So swapping matplotlib for Inventor is, by design, an **adapter swap**. The contract holds:

```
generate(params: dict) -> { ...figures..., "drawing_file": "<file the GUI can show>" }
```

The MCP tool signatures (`update_silencer_design(project_id, length_mm, ...)`), the ENGINES dispatch table, the validation rules, and the human-confirm gate all stay the same. Only the body of `generate()` changes: instead of drawing with matplotlib, it drives Inventor and exports a drawing (PDF / PNG / DWG / STEP). That isolation is the actual answer to this question.

2 An important distinction: geometry vs. physics

The current Python engine does **two** jobs: it computes acoustic figures (peak TL, NRC, absorption) and it renders a drawing. Inventor and AutoCAD produce **geometry and drawings, not acoustics** — CAD does not know what a transmission-loss curve is. The clean split is therefore:

- **Keep the acoustic calculation in Python** — it is a handful of deterministic formulas (the `_derive` step in each engine). This is the cheap, fast part and there is no reason to move it.
- **Use Inventor only for the geometry and manufacturing drawing** — Inventor regenerates the parametric 3D model and exports a proper engineering drawing.

So “the engine is Inventor” really means Inventor replaces the *drawing* half. The acoustic numbers stay where they are. Stating this up front shrinks the integration — you are not asking Inventor to do something it cannot.

3 Prerequisite: a parametric master model

Today `generate()` computes geometry from formulas on the fly. Inventor needs a **pre-built parametric model** (an `.ipt` part or `.iam` assembly, with an `.idw/.dwg` drawing) authored once by an engineer, whose **user parameters** are named to map onto our keys:

<code>length_mm</code>	-> <code>d_Length</code>	<code>rockwool_mm</code>	-> <code>d_Rockwool</code>
<code>width_mm</code>	-> <code>d_Width</code>	<code>epdm_mm</code>	-> <code>d_EPDM</code>
<code>height_mm</code>	-> <code>d_Height</code>	<code>steel_mm</code>	-> <code>d_Steel</code>

The integration then becomes: set these user parameters, force a model update, export the drawing, and read back any measured properties. The splitter layout ($W/8 + W/4 + \dots$) would live as relations or iLogic inside that model instead of as Python.

4 Ways to drive Inventor

Listed roughly from “quick local demo” to “robust production”, with trade-offs.

4.1 Local COM / .NET automation (pywin32)

Inventor exposes a full COM automation API. From Python on Windows you can `Dispatch("Inventor.Application")`, open the master model, set user parameters, call `Update`, and export the drawing. **Pros:** direct, full control, no cloud; fastest path to a working on-prem version. **Cons:** Windows-only; Inventor must be installed, licensed and running; COM is finicky; starting Inventor per request is slow.

4.2 iLogic rules (inside the model)

iLogic is Inventor's built-in VB.NET-flavoured scripting. Rules authored in the model recompute everything from the parameters; the external driver just sets parameters and triggers the rule. **Pros:** engineering logic lives with the model; robust regeneration. **Cons:** you maintain VB.NET inside a CAD file, and still need something to invoke it (usually 4.1 or 4.5).

4.3 An Inventor add-in exposing a local HTTP endpoint

A small C#/.NET add-in runs an HTTP server inside the Inventor process; the MCP tool POSTs the new parameters and gets back the exported drawing path. **Pros:** keeps one Inventor instance **warm** (no per-call startup cost), clean request/response, language-agnostic caller — it matches our existing “call a service” model nicely. **Cons:** you maintain a .NET add-in, and still need one Windows box with a licensed Inventor.

4.4 Autodesk Platform Services — Design Automation API (recommended for production)

APS (formerly Forge) runs **Inventor headless in Autodesk's cloud**. You upload the parametric model plus an AppBundle and Activity, then submit a work item with the new parameter values; Autodesk runs Inventor server-side and returns the exported derivatives (PDF/DWG/STEP/PNG). **Pros:** no Windows infrastructure to babysit, no local licence to manage, scales horizontally, OS-independent (our Linux container stays a Linux container). **Cons:** cloud dependency and per-use cost; an **asynchronous** job model (submit then poll), so `generate()` becomes `async` with a status wait; you package the model and activity once.

4.5 File / watched-folder exchange

Write the parameters to a JSON or Excel file that a running Inventor instance (via an iLogic batch rule) watches; Inventor regenerates and drops a PDF/PNG back into an output folder. **Pros:** dead simple, fully decouples languages, easy to debug. **Cons:** crude, polling-based, fragile around concurrency and partial writes — fine for a prototype, not a product.

5 What about AutoCAD specifically?

AutoCAD has the same shape of options — COM/ActiveX automation, the .NET API, AutoLISP scripts, and **APS Design Automation for AutoCAD** (a headless AutoCAD core engine in the cloud running scripts against DWGs). But AutoCAD is primarily a **2D drafting** tool; for parametric *mechanical* parts with real 3D build-up — which is exactly what splitters and stratified panels are — **Inventor is the better fit**. If the deliverable were 2D layout drawings only, AutoCAD plus Design Automation would be the lighter choice.

6 How generate() changes — and what does not

```
# unchanged: agent loop, MCP tools, store validation, propose/commit, email flow
# unchanged contract:
generate(params) -> { figures..., "drawing_file": "..."}

# changed: the body
#   was: matplotlib draws a PNG
#   now: 1. compute acoustic figures in Python      (unchanged formulas)
#         2. push params into the Inventor master model (COM / add-in / APS)
#         3. trigger update + export drawing (PDF/PNG/DWG/STEP)
#         4. return the exported file as drawing_file
```

The store would gain an `ENGINES` entry pointing at an `inventor_adapter` instead of `engine`, and the `silencer/panel split` stays exactly as it is — each product simply maps to a different master model. The one structural change worth flagging: on the cloud (APS) route, regeneration is asynchronous and slower (seconds, not milliseconds), so the chat path would acknowledge (“regenerating in Inventor...”) and the drawing arrives when the job finishes. The email path already handles this gracefully — it parks a proposal for later human review — so it barely changes.

7 Recommendation

- **For a robust, deployable system** — APS Design Automation for Inventor (4.4). Keep the acoustic physics in Python; let cloud-Inventor own the CAD geometry and drawings. No Windows servers to maintain, it scales, and the existing Linux/Docker setup is untouched.
- **For a quick on-prem demo** — a warm Inventor instance driven by a small add-in HTTP endpoint (4.3), or plain COM automation (4.1) if time is tight.
- **In every case** — the MCP tool boundary stays fixed. That is the whole reason the agent, the schema handling, and the email pipeline do not have to know or care that a real CAD kernel replaced matplotlib.